

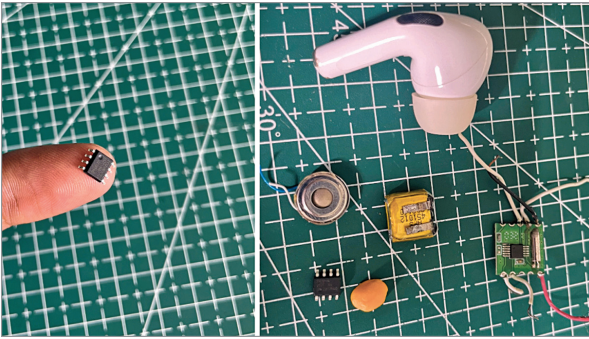


Fig. 2: The Tiny412 MCU IC shown on a fingertip, with the sizes of the remaining components compared to a pulse seed

```

1  RDA_setVolume(volume);
2
3  // Loop
4  while(1) {
5      uint16_t pressTime = 0;
6      while(!pinRead(PIN_SWITCH)) {          // Button pressed
7          _delay_ms(10);
8          pressTime += 10;
9          if(pressTime == 2000) {             // 2s hold: Seek down
10             RDA_seekDown();
11             RDA_waitTuning();
12             channel = RDA_channel;
13             EEPROM_update();
14         }
15         if(pressTime == 10000) {             // 10s hold: Increase volume
16             if(volume < 15) volume++;
17             RDA_setVolume(volume);
18             EEPROM_update();
19         }
20         if(pressTime == 30000) {             // 30s hold: Decrease volume
21             if(volume > 0) volume--;
22             RDA_setVolume(volume);
23             EEPROM_update();
24         }
25     }
26     if(pressTime > 0 && pressTime < 2000) {   // Quick press (<2s): Seek up
27         RDA_seekUp();
28         RDA_waitTuning();
29         channel = RDA_channel;
30         EEPROM_update();
31     }
32 }

```

Fig. 3: Code snippet defining button functions for the user interface (UI)

has been used, one of the smallest available. It communicates via I2C, allowing channel changes, volume control, and station scanning, all managed through an MCU.

The selection of the MCU is particularly critical. The voltage operating range of the MCU must match that of the FM module. The MCU must also provide an I2C port for communication with the FM module and consume the smallest amount of energy to maximise the runtime on a small battery. To meet these requirements, one recently released option is from Texas Instruments (TI), detailed at:

<https://www.ti.com/about-ti/newsroom/news-releases/2025/2025->

03-11-ti-introduces-the-world-s-smallest-mcu-enabling-innovation-in-the-tiniest-of-applications.html.

Another option is from Microchip's ATtiny series, which offers SMD versions in various small packages. The WCH CH32V003 series of MCU chips can also be considered.

For ease of programming, the ATtiny412 MCU from Microchip was selected. However, other MCUs can also be used with appropriate code adjustments. The ATtiny412 requires a UPDI programmer but can also be programmed using an Arduino Uno, which many hobbyists already have. Similarly, other MCUs such as the ATtiny85 can be programmed

via Arduino Uno. In this device, the Arduino Uno is used as a programmer board, not as the main MCU. Fig. 2 displays the Tiny412 MCU IC on a fingertip, alongside other components compared to a pulse seed for scale.

To assemble this device, components are listed in the Bill of Materials table.

Creating firmware

The code has been prepared using the Arduino IDE. It is natural to wonder whether this MCU can be programmed using the IDE. The answer is yes, not just this model, but all the MCUs mentioned earlier are compatible. To keep the design

as compact and simple as possible, with the battery, buttons, and PCB all fitting inside the earbud, a single pushbutton has been used to handle the entire user interface. This one switch manages scanning for stations, changing channel frequencies, adjusting volume, and play/pause control.

A common question is how all of these functions can be managed with a single switch.

The working of the switch and its function definitions in the code are explained next. Fig. 3 shows the code snippet defining button functions for the user interface (UI).

Switch functions.

Quick press (< 2 seconds).

- Function: Seek the next radio channel

- How it works: A quick press and release within 2 seconds triggers the `RDA\_seekUp()` function, which tunes to the next available station. The updated channel is saved to EEPROM.

- Code reference: if (pressTime > 0 && pressTime < 2000) in the main loop.

Press and hold for 2 seconds.

- Function: Seek the previous radio channel

- How it works: Holding the button for exactly 2 seconds activates the `RDA\_seekDown()` function, switching to the previous station. The updated channel is saved to EEPROM.

- Code reference: if (pressTime == 2000) in the main loop

Long press (10 seconds).

- Function: Increase volume
- How it works: Holding the button for 10 seconds initiates volume increase (up to a maximum of 15). The new volume is set via `RDA\_setVolume()` and saved to EEPROM. Volume continues to increase incrementally while the button remains pressed beyond the 10-second mark.